

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieure et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene



Mini Projet

Pipeline simplifié d'un encodeur vidéo MPEG-4

Travail présenté à M **Abadli**
Module : **TP Multimédia**
Spécialité : **IL M1**

Présenté par :
Manel Lahouel
BOUDA Hadjer Rania

Pour le : 20 mai 2026

I. Introduction :

La compression vidéo est au cœur des systèmes multimédias modernes. Dans ce projet, nous avons implémenté un pipeline simplifié inspiré du standard MPEG-4, permettant de compresser une séquence de frames JPG en un fichier binaire reconstituable. Le pipeline couvre cinq étapes : conversion colorimétrique, codage intra-image (I-frames), codage inter-images (P-frames), codage entropique, et évaluation. Sur 30 frames de 320×180 pixels, le système atteint un taux de compression de 15,21×.

II. Description du pipeline :

1. Étape 1 : Pré-traitement : Conversion colorimétrique et sous-échantillonnage

La première étape consiste à convertir chaque frame du format BGR (natif OpenCV) vers l'espace colorimétrique YCbCr, qui sépare la luminance (Y) des composantes de chrominance (Cb, Cr). Cette séparation est fondamentale : l'œil humain est bien plus sensible aux variations de luminosité qu'aux variations de couleur.

On ensuite appliqué un sous-échantillonnage 4:2:0 sur les canaux Cb et Cr en ne conservant qu'un pixel sur deux dans chaque direction spatiale. Cela réduit la résolution de la chroma de moitié en largeur et en hauteur, sans perte visible à l'œil nu.

Résultats obtenus pour chaque frame :

```
Étape 1 : 30 frames converties
Y  : (180, 320) ← pleine resolution
Cb : (90, 160)  ← divise par 2 (4:2:0)
Cr : (90, 160)  ← divise par 2 (4:2:0)
```

Cette étape seule réduit déjà le volume de données à traiter de 50 % pour la chrominance.

2. Étape 2 : Codage intra-image (I-frames) :

Les I-frames sont codées de manière indépendante, sans référence à d'autres frames. Le codage repose sur la Transformée en Cosinus Discrète 2D (DCT-2D) appliquée bloc par bloc sur des fenêtres 8×8 pixels.

Le processus complet pour chaque canal et chaque bloc est le suivant :

- Centrage : soustraction de 128 à chaque pixel (décalage vers des valeurs signées)
- DCT 2D : transformation du bloc spatial en domaine fréquentiel
- Quantification : division des coefficients par la matrice Q, arrondi à l'entier le plus proche (introduit la perte)
- Stockage des coefficients quantifiés sous forme int16

La matrice de quantification utilisée suit la formule du cours $Q(i,j) = 1 + (1 + i + j) \times F_q$, avec $F_q = 5$. Elle pénalise davantage les hautes fréquences (en bas à droite) que les basses fréquences (DC et proches), ce qui correspond au comportement perceptif humain.

Matrice Q obtenue pour $F_q = 5$:

```

Étape 2 : fonctions DCT definies | Q_MATRIX (Fq=5) :
[ 6 11 16 21 26 31 36 41]
[11 16 21 26 31 36 41 46]
[16 21 26 31 36 41 46 51]
[21 26 31 36 41 46 51 56]
[26 31 36 41 46 51 56 61]
[31 36 41 46 51 56 61 66]
[36 41 46 51 56 61 66 71]
[41 46 51 56 61 66 71 76]]

```

Le décodeur inverse le processus : dequantification (multiplication par Q) puis IDCT-2D, suivi d'un reclampage dans [0, 255] et ajout de 128.

3. Étape 3 : Codage inter-images (P-frames) :

Les P-frames exploitent la redondance temporelle entre frames consécutives. C'est là que réside l'essentiel du gain de compression : les frames d'une vidéo sont souvent très similaires les unes aux autres.

Architecture GOP (Group of Pictures) :

- Toutes les 10 frames (GOP_SIZE = 10), une I-frame est insérée.
- Les 9 frames suivantes sont des P-frames codées par référence à la frame précédente reconstruite.

Pour chaque P-frame, le canal Y est découpé en macroblocs de 16×16 pixels. Pour chaque macrobloc, je recherche le bloc le plus similaire dans la frame de référence, dans une fenêtre de 8 pixels (SEARCH_WINDOW = 8). La similarité est mesurée par l'erreur quadratique moyenne (MSE).

Une fois le meilleur vecteur de mouvement (dx, dy) identifié, on calcule le résidu : différence entre le bloc courant et sa prédiction. Ce résidu généralement très faible est ensuite encodé par DCT + quantification, exactement comme pour une I-frame.

Le décodeur reconstruit chaque P-frame en combinant la prédiction (déplacement du bloc de référence) avec le résidu décodé.

Résultats :

```

Étape 3 : fonctions P-frames definies
Frame 000 → I-frame
Frame 001 → P-frame
Frame 002 → P-frame
Frame 003 → P-frame
Frame 004 → P-frame
Frame 005 → P-frame
Frame 006 → P-frame
Frame 007 → P-frame
Frame 008 → P-frame
Frame 009 → P-frame
Frame 010 → I-frame
Frame 011 → P-frame
Frame 012 → P-frame
Frame 013 → P-frame
Frame 014 → P-frame
Frame 015 → P-frame
Frame 016 → P-frame
Frame 017 → P-frame
Frame 018 → P-frame
Frame 019 → P-frame
Frame 020 → I-frame
Frame 021 → P-frame
Frame 022 → P-frame
Frame 023 → P-frame
Frame 024 → P-frame
Frame 025 → P-frame
Frame 026 → P-frame
Frame 027 → P-frame
Frame 028 → P-frame
Frame 029 → P-frame
Encodage termine : 3 I-frames + 27 P-frames

```

4. Étape 4 : Codage entropique :

Une fois toutes les frames encodées (coefficients DCT quantifiés + vecteurs de mouvement), l'ensemble de la structure est sérialisé avec le module pickle de Python, puis compressé via zlib au niveau maximal (level=9).

zlib utilise l'algorithme DEFLATE, combinaison de LZ77 (élimination des répétitions) et de codage de Huffman (allocation de codes courts aux symboles fréquents). C'est une approche de compression sans perte parfaitement adaptée à des données hautement structurées comme des coefficients entiers.

Résultats de compression :

```
Étape 4
Taille originale : 5062 KB
Taille compressée : 333 KB
Ratio compression : 15.21x
Decodage vérifié : 30 frames
```

5. Étape 5 : Évaluation & Visualisation :

On a produit une figure récapitulant toutes les étapes du pipeline. Elle contient les métriques calculées incluent le taux de compression global, la répartition I-frames / P-frames et la visualisation des résidus.

III. Choix de conception et justification :

1. Choix de l'espace colorimétrique YCbCr :

Le passage de BGR à YCbCr est motivé par la physiologie de la vision humaine. Les cônes de l'œil sont plus denses dans les zones de forte luminosité, et le cortex visuel est beaucoup plus sensible aux contrastes de luminance qu'aux variations de teinte. En séparant Y des composantes chroma, on peut appliquer une compression bien plus agressive sur Cb et Cr sans dégradation perceptible c'est le principe du sous-échantillonnage 4:2:0.

2. Taille des blocs DCT : 8×8 pixels:

La taille 8×8 est un compromis bien connu entre complexité computationnelle et efficacité de compression. Des blocs plus grands captureraient plus de corrélations spatiales mais seraient plus coûteux à calculer. Des blocs plus petits nécessiteraient davantage d'appels DCT. Le 8×8 est le standard JPEG/MPEG depuis les années 1990 et reste le bon choix pour ce projet.

3. Facteur de quantification $F_q = 5$:

La formule $Q(i,j) = 1 + (1 + i + j) \times F_q$ génère une matrice qui pénalise progressivement les hautes fréquences. Avec $F_q = 5$, le coefficient DC (0,0) vaut 6 et le coefficient haute fréquence (7,7) vaut 76. Cela signifie que les détails fins sont fortement compressés, tandis que la structure globale de l'image (basses fréquences) est préservée avec plus de fidélité. Un F_q plus élevé donnerait une meilleure compression mais une qualité visuelle dégradée.

4. Taille des macroblochs 16×16 et fenêtre de recherche 8 :

Les macroblochs 16×16 sont la taille standard en MPEG pour l'estimation de mouvement. Une fenêtre de 8 pixels couvre les mouvements modérés, typiques d'une vidéo de marche à pied filmée depuis une position fixe. Une fenêtre plus grande augmenterait la qualité de prédiction mais au prix d'un coût computationnel quadratiquement plus élevé.

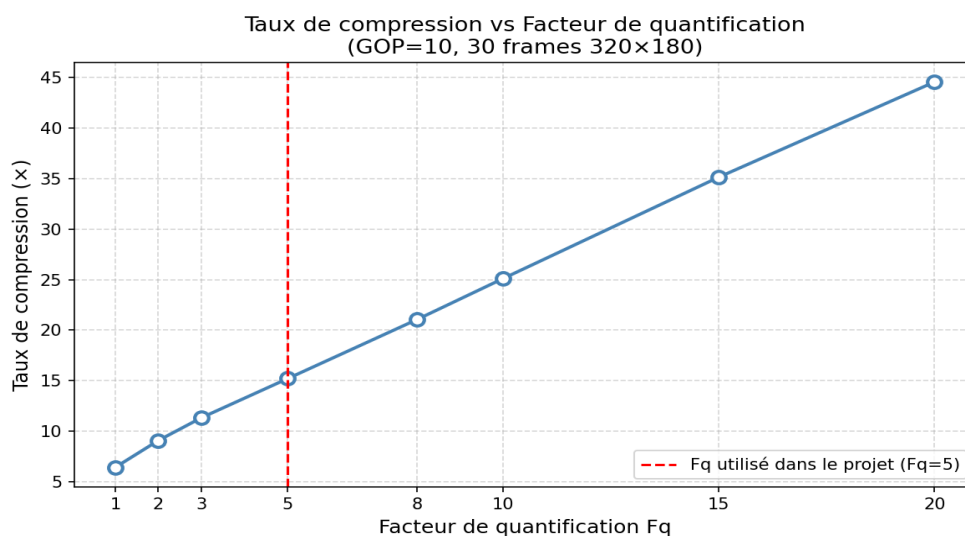
5. GOP de taille 10 :

Un GOP de 10 frames signifie qu'une I-frame est insérée toutes les 10 frames. Pour 30 frames, cela donne 3 I-frames et 27 P-frames. C'est un ratio raisonnable : les I-frames sont plus volumineuses mais permettent de resynchroniser le décodeur et de limiter la propagation des erreurs. Un GOP plus grand améliore le taux de compression mais rend le seeking plus difficile et amplifie les artefacts en cas d'erreur.

IV. Analyse expérimentale :

1. Impact du facteur de quantification sur le taux de compression :

Pour mesurer l'effet de F_q , on a ré-encodé la vidéo complète pour chaque valeur de F_q en gardant $\text{GOP} = 10$ fixe.

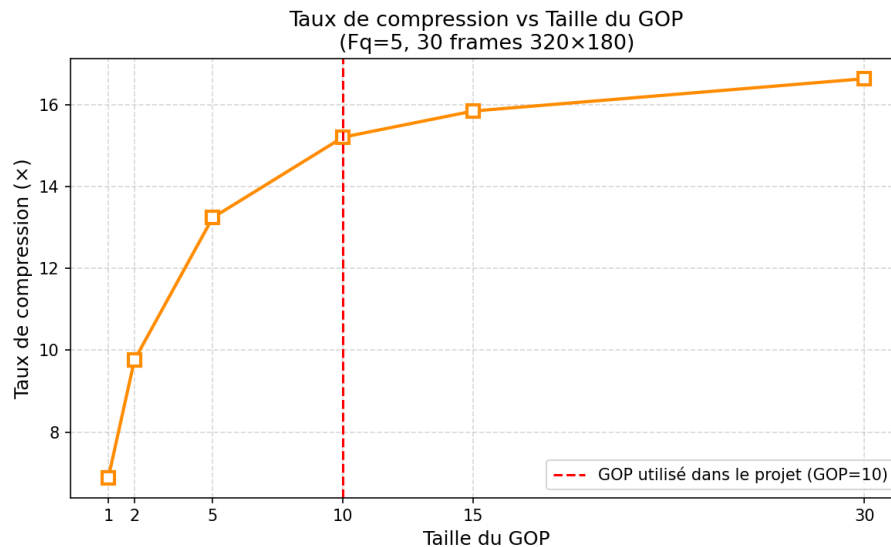


La courbe montre une relation quasi-linéaire entre F_q et le taux de compression sur notre séquence ce qui s'explique par la nature de la matrice Q utilisée : $Q(i,j) = 1 + (1+i+j) \times F_q$. Quand F_q double, les coefficients DCT sont divisés par des valeurs environ deux fois plus grandes, ce qui force deux fois plus de coefficients à s'arrondir à zéro. Ces zéros sont ensuite très efficacement comprimés par zlib grâce à l'algorithme LZ77.

Et on a choisi la valeur $F_q = 5$ comme point d'équilibre car le taux de $15,2\times$ est excellent, et les frames reconstruites restent visuellement proches des originales à l'œil nu.

2. Impact de la taille du GOP sur la compression :

Pour mesurer l'effet du GOP, on a réencodé la vidéo complète pour chaque valeur de GOP en gardant $F_q = 5$ fixe.



La courbe présente une forme en plateau logarithmique très claire : le gain est important entre $GOP=1$ et $GOP=10$, puis devient marginal au-delà. Cela s'explique ainsi : passer de tout I-frames ($GOP=1$) à un mix I+P ($GOP=10$) est très bénéfique car les P-frames exploitent la forte similarité temporelle entre frames consécutives. Mais au-delà de $GOP=10$, les frames supplémentaires sont déjà si bien prédites que leur résidu est déjà quasi nul il n'y a plus grand chose à gagner.

$GOP = 10$ a donc été retenu car il capture l'essentiel du gain de compression tout en conservant une robustesse correcte : en cas d'erreur de décodage, la resynchronisation intervient au plus tous les 10 frames.

3. Observation sur les vecteurs de mouvement :

La visualisation des vecteurs de mouvement confirme pourquoi les P-frames sont si efficaces sur cette séquence. On observe que la quasi-totalité des macrobloccs sont représentés par des points verts (vecteur nul) : le fond forestier ne bouge pas d'une frame à l'autre, et la prédiction depuis la frame précédente est parfaite sans aucun déplacement. Seule la silhouette du personnage génère des flèches jaunes, indiquant un déplacement latéral cohérent avec son mouvement de marche.

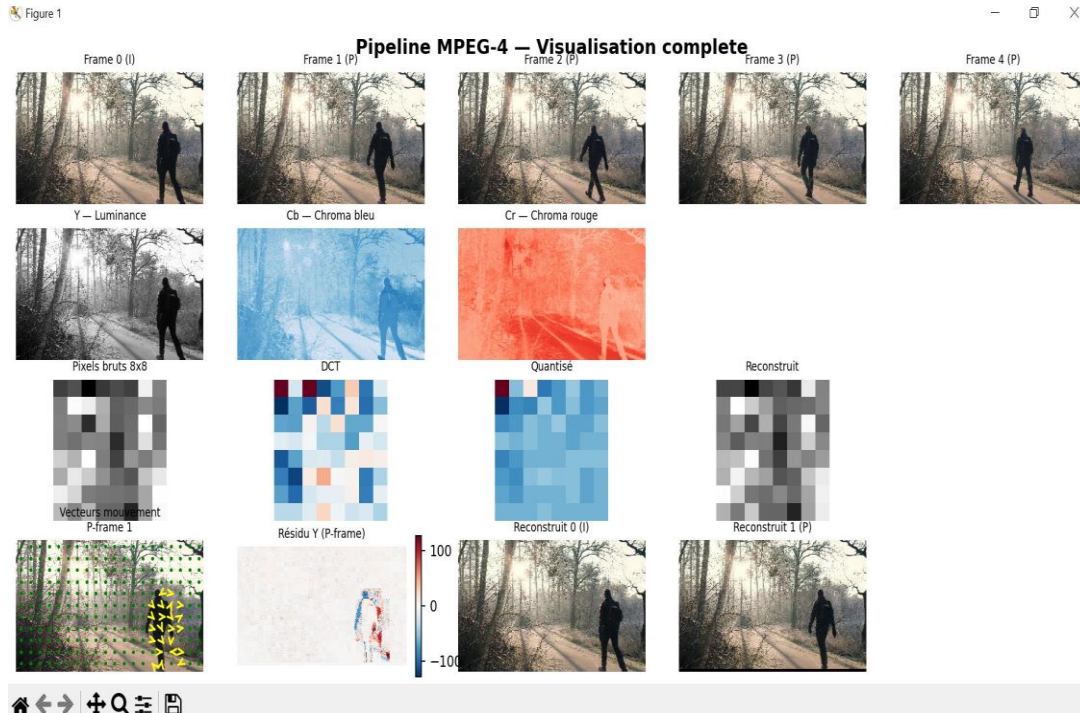
Conséquence directe sur la compression : les résidus sont quasi nuls sur 90% de la frame seule une petite région autour du personnage nécessite un codage non trivial. C'est ce qu'on observe clairement dans la carte de résidu Y de la figure du pipeline, où seul le contour du personnage apparaît en rouge/bleu, tout le reste étant neutre.

V. Résultats et visualisation du pipeline :

On a produit une figure récapitulant toutes les étapes du pipeline

La figure ci-dessous, elle est organisée en quatre lignes :

- Ligne 1 : Frames originales sont les 5 premières frames (frame 0 en I, frames 1-4 en P).
- Ligne 2 : Canaux YCbCr sont Y en niveaux de gris, Cb en bleu, Cr en rouge, montrant la séparation luminance/chrominance.
- Ligne 3 : Transformation DCT sur un bloc 8×8 : pixels bruts $\rightarrow$ coefficients DCT $\rightarrow$ coefficients quantifiés $\rightarrow$ reconstruction.
- Ligne 4 : Vecteurs de mouvement, résidu Y, frames reconstruits.



Observations :

- Le canal Y capture fidèlement la structure de la scène (silhouette, arbres, chemin).
- Le canal Cb montre une dominance bleue uniforme (ciel et brume), le canal Cr capte les tons chauds du sol.
- Le bloc DCT 8×8 montre que la quantification efface presque entièrement les hautes fréquences le résultat reconstruit reste visuellement proche de l'original.
- Le résidu Y de la P-frame est quasi nul partout sauf au niveau du personnage en mouvement confirmant l'efficacité de la prédiction inter-frames.
- Les frames reconstruites (I et P) sont visuellement indiscernables des originales à l'œil.

VI. Conclusion :

Ce projet nous a permis d'implémenter de bout en bout un codec vidéo simplifié reproduisant les mécanismes fondamentaux de MPEG-4. Les résultats sont concluants : un taux de compression de $15,21 \times$ avec une qualité visuelle préservée. L'analyse expérimentale confirme que $GOP=10$ et $Fq=5$ représentent des choix bien équilibrés.